

T1 回文串

题意简化

输入长度 $1 \leq n \leq 10^6$ 的小写字母串 s 。可任意重排字符，要求非空连续回文子串的最大数量；即使内容相同，不同起止位置也分别计数。输出最大数量。

问题拆分

1. 按字符统计出现次数，给每个回文子串的端点归类。
2. 求端点对给出的上界，并构造能使每个上界都达到的排列。

部分分（累计 30 分， $|s| \leq 3$ ）：枚举不同排列

1. 先排序字符串，再用 `next_permutation` 枚举至多 $n!$ 种不同排列，排序花 $O(n \log n)$ 。
2. 每个排列枚举 $O(n^2)$ 个非空区间，每区间双指针判回文 $O(n)$ ，计数并更新最大值。

总时间 $O(n \log n + n!n^3)$ 、额外空间 $O(n)$ ； $n \leq 3$ 时可直接完成。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    sort(s.begin(),s.end());
    int n = (int)s.size();
    long long ans = 0;
    do{
        long long count = 0;
        for(int l = 0; l < n; l++){
            for(int r = l; r < n; r++){
                bool same = true;
                int i = l, j = r;
                while(i < j){
                    if(s[i] != s[j]) same = false;
                    i++;
                    j--;
                }
                if(same) count++;
            }
        }
        ans = max(ans, count);
    }while(next_permutation(s.begin(),s.end()));
    cout << ans << '\n';
    return 0;
}
```

满分做法：相同字符连续分块

长度不超过 3 的累计 30 分可用下方排列枚举；其余 70 分用频次公式。排列枚举在 $n = 10^6$ 时无法运行。

1. 扫描 n 个字符统计 26 个频次， $O(n)$ 时间、 $O(26)$ 空间。
2. 某字符出现 c 次，它能贡献的单点与同字符端点对至多 $c + \binom{c}{2} = c(c+1)/2$ ；26 类各算一次 $O(26)$ 。把同字符连续排列，块内任一端点对之间全相同，全部达到上界。

总时间 $O(n + 26)$ 、空间 $O(26)$ ；答案最大约 $n(n+1)/2$ ，须用 `long long`。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    long long cnt[26] = {};
    for(int i = 0; i < (int)s.size(); i++){
```

```
    cnt[s[i] - 'a']++;  
}  
long long ans = 0;  
for(int i = 0; i < 26; i++){  
    // 将相同字符放在一起, 该块贡献 1+2+...+cnt[i].  
    ans += cnt[i] * (cnt[i] + 1) / 2;  
}  
cout << ans << '\n';  
return 0;  
}
```

知识点总结

重排后最大化局部相等结构时，先用相同字符端点对限制答案，再让相同字符相邻以达到上界。

T2 交错四元组

题意简化

输入 $4 \leq n \leq 2000$ 的整数数组 a ，元素在 $[-10^9, 10^9]$ 内，可相等或为负。统计所有递增下标 $i < j < k < l$ 且严格满足 $a_i < a_k < a_j < a_l$ 的四元组数量，按下标区别方案并输出。

问题拆分

1. 固定中间下标 j, k ，先检查 $a_k < a_j$ 。
2. 分别数左侧 $i < j$ 且 $a_i < a_k$ 、右侧 $l > k$ 且 $a_l > a_j$ 的位置。
3. 两侧独立组合，累加各中间对的乘积。

部分分（累计 20 分， $n \leq 20$ ）：四重枚举

1. 读入数组， $O(n)$ 时间、空间。
2. 枚举 $i < j < k < l$ ，共 $\binom{n}{4}$ 组；每组 $O(1)$ 检查三个严格不等式并加一。

总时间 $O(n^4)$ 、空间 $O(n)$ ； $n = 20$ 仅 4845 组。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

int a[2005];

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    long long ans = 0;
    for(int i = 1; i <= n; i++){
        for(int j = i + 1; j <= n; j++){
            for(int k = j + 1; k <= n; k++){
                for(int l = k + 1; l <= n; l++){
                    if(a[i] < a[k] && a[k] < a[j] && a[j] < a[l]){
                        ans++;
                    }
                }
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（累计 50 分， $n \leq 800$ ）：三重计数

1. 对每个 j 从右往左计算各 k 右侧大于 a_j 的位置数，每个 j 花 $O(n)$ ，共 $O(n^2)$ 。
2. 枚举 $i < j < k$ 并检查 $a_i < a_k < a_j$ ，每组 $O(1)$ ；成立时加对应右侧数量，共 $O(n^3)$ 。

总时间 $O(n^3)$ 、空间 $O(n)$ ； $n = 800$ 时约检查一亿组三下标，1 秒测试机上实测可通过。 $n = 2000$ 的一般数据需要换用平方级方法。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

int a[2005], greaterRight[2005];

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
    long long ans = 0;
```

```

for(int j = 2; j <= n - 2; j++){
    greaterRight[n] = 0;
    for(int k = n - 1; k > j; k--){
        greaterRight[k] = greaterRight[k + 1];
        if(a[k + 1] > a[j]){
            greaterRight[k]++;
        }
    }
    for(int i = 1; i < j; i++){
        for(int k = j + 1; k < n; k++){
            if(a[i] < a[k] && a[k] < a[j]){
                ans += greaterRight[k];
            }
        }
    }
}
cout << ans << '\n';
return 0;
}

```

满分做法：固定中间两项的平方级计数

公开分档对应：1~2 点 $n \leq 20$ (20 分) 用四重枚举；3~5 点 $n \leq 800$ (新增 30 分) 用三重计数，连同前两点共 50 分；6~10 点 (其余 50 分) 用平方级程序。第 3~5 点接近 800 个元素，四重枚举不能在时限内通过。

1. 预处理 $leftLess[j][k]$ ：固定 k ，让 j 从左到右增长并累积小于 a_k 的左侧元素； $O(n^2)$ 时间、空间。
2. 对每个 j 从右向左扫 k ，维护右侧大于 a_j 的位置数， $O(n^2)$ 次 $O(1)$ 更新；先计当前 k ，再把它加入右侧，保证 $l > k$ 。
3. 仅当 $a_k < a_j$ 时加 $leftLess[j][k]$ 乘右侧数量， $O(1)$ ；所有中间对共 $O(n^2)$ 。

总时间 $O(n^2)$ 、空间 $O(n^2)$ ， $n = 2000$ 时约四百万表项。严格比较使相等元素不被误计。

参考代码

```

#include<bits/stdc++.h>
using namespace std;

int a[2005], leftLess[2005][2005];

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
    // leftLess[j][k]: 下标小于 j、数值小于 a[k] 的元素数。
    for(int k = 1; k <= n; k++){
        for(int j = 2; j < k; j++){
            leftLess[j][k] = leftLess[j - 1][k];
            if(a[j - 1] < a[k]){
                leftLess[j][k]++;
            }
        }
    }
    long long ans = 0;
    for(int j = 2; j <= n - 2; j++){
        long long rightGreater = 0;
        for(int k = n; k > j; k--){
            if(a[k] < a[j]){
                ans += leftLess[j][k] * rightGreater;
            }
            // 更新放在计数之后，保证第四个位置严格大于 k。
            if(a[k] > a[j]){
                rightGreater++;
            }
        }
    }
    cout << ans << '\n';
    return 0;
}

```

知识点总结

四元组的条件可由固定中间两项分离为左右独立计数时，预处理一边、扫描另一边，比四重枚举更稳。

T3 文件判定

题意简化

输入 $t \leq 100$ 组父子关系，每组至多 10 条，关系 $A \ B$ 表示条目 B 是 A 的直接孩子，顺序任意，同名表示同一条目。判断是否恰好是一棵三层目录树：唯一根名为非空字母后接非空数字；根下恰好四个名称互异的纯字母题目条目；每题下恰有一个名为“题名.cpp”的叶条目。大小写敏感，不允许多余条目或关系，每组输出 yes 或 no。

问题拆分

1. 确定唯一根，并核查根名称及总关系数。
2. 找到根的直接孩子，检查其名称和各自唯一文件名。
3. 确认所有给定关系均已被这八条合法边消耗。

满分做法：逐项核对关系

公开分档对应：1~2 点同时满足性质 1、2、3；3~4 点满足性质 1、3；5~6 点满足性质 2；7~10 点无额外性质。四档均由本节的 8 条关系完整核验代码处理：性质只预先保证部分合法条件，仍须检查其余条件，因此不另写部分分代码。

1. 每组先要求恰好 8 条关系；在至多 n 个名称中找作为父但不作为子者并验证唯一根及字母数字格式，朴素比较 $O(n^2)$ 。
2. 扫描关系找根下四个互异纯字母题目名；再分别找它们唯一的 题名.cpp 叶子，最多 $O(n^2)$ 。
3. 八条合法关系已恰好占满输入；再核查没有额外连接或重复， $O(n^2)$ 。各组独立清理状态。

每组总时间 $O(n^2)$ 、空间 $O(n)$ ； $n \leq 10$ ，直接检查足够。名称大小写原样比较，不能按路径顺序假设输入已排序。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

string a[11], b[11];
int n;

bool isLetter(char c){
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

bool rootNameOK(const string &s){
    int i = 0;
    while(i < (int)s.size() && isLetter(s[i])){
        i++;
    }
    if(i == 0 || i == (int)s.size()){
        return false;
    }
    while(i < (int)s.size()){
        if(s[i] < '0' || s[i] > '9'){
            return false;
        }
        i++;
    }
    return true;
}

bool check(){
    if(n != 8){
        return false;
    }
    string root = "";
    for(int i = 1; i <= n; i++){
        bool hasParent = false;
        for(int j = 1; j <= n; j++){
            if(a[i] == b[j]){
                hasParent = true;
            }
        }
        if(!hasParent){
            if(root != "" && root != a[i]){
                return false;
            }
            root = a[i];
        }
    }
    if(!rootNameOK(root)){
        return false;
    }
    int tasks = 0;
    for(int i = 1; i <= n; i++){
        if(a[i] != root){
            tasks++;
        }
    }
    return tasks == 4;
}
```

```

        continue;
    }
    tasks++;
    string task = b[i];
    for(int j = 0; j < (int)task.size(); j++){
        if(!isLetter(task[j])){
            return false;
        }
    }
    int files = 0;
    for(int j = 1; j <= n; j++){
        if(a[j] == task){
            files++;
            if(b[j] != task + ".cpp"){
                return false;
            }
        }
    }
    if(files != 1){
        return false;
    }
}
// 四条根边 + 四条代码文件边已经占满全部八条关系。
return tasks == 4;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while(t--){
        cin >> n;
        for(int i = 1; i <= n; i++){
            cin >> a[i] >> b[i];
        }
        if(check()){
            cout << "yes\n";
        }else{
            cout << "no\n";
        }
    }
    return 0;
}

```

知识点总结

小规模关系判定题可按必要条件逐层核对，并用总边数与唯一性证明没有遗漏的多余条目。

T4 树与叶子

题意简化

输入 $T \leq 10$ 棵无根树，每棵 $2 \leq n \leq 10^5$ 。删掉任意一条边后得到两部分，按删边后的度数分别数叶子：度不超过 1 的点都是叶子，孤点也算。求两部分叶子数差的最小绝对值及达到它的边数，每棵树输出一行。

问题拆分

1. 统计原树叶子总数，并为每条边求一侧原叶子数。
2. 根据删边两端的原度数，修正两侧新增的叶子。
3. 对每条边计算新叶子数差，维护最小值与方案数。

部分分（小规模及链形，共 30 分）：逐边断开

1. 读入邻接表与原度数， $O(n)$ 时间、空间。
2. 对每条边临时禁止通行，从两端分别 BFS，按删边后的度数统计各部分叶子；每条边 $O(n)$ 。
3. 若原树最大度不超过 2，它是一条链。 $n = 2$ 时答案为 $(0, 1)$ ； $n = 3$ 时为 $(1, 2)$ ； $n \geq 4$ 时断开内部边得到两侧各两个叶子，答案为 $(0, n - 3)$ 。判链及公式计算共 $O(n)$ 。
4. 非链时比较两侧叶子数差，更新最小值与方案数；共 $n - 1$ 条边，每条 $O(1)$ 。

一般树时间 $O(n^2)$ 、空间 $O(n)$ ， $n \leq 1000$ 可行；链形时间 $O(n)$ ，可覆盖独立的长链测试点。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

vector<int> g[100005];
int edgeU[100005], edgeV[100005], que[100005];
bool seen[100005];
int n, bannedU, bannedV;

int countLeaves(int start){
    for(int i = 1; i <= n; i++){
        seen[i] = false;
    }
    int head = 0, tail = 1, result = 0;
    que[0] = start;
    seen[start] = true;
    while(head < tail){
        int u = que[head++];
        int degree = (int)g[u].size();
        if(u == bannedU || u == bannedV){
            degree--;
        }
        if(degree <= 1){
            result++;
        }
        for(int j = 0; j < (int)g[u].size(); j++){
            int v = g[u][j];
            if((u == bannedU && v == bannedV) ||
               (u == bannedV && v == bannedU)){
                continue;
            }
            if(!seen[v]){
                seen[v] = true;
                que[tail++] = v;
            }
        }
    }
    return result;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while(t--){
        cin >> n;
        for(int i = 1; i <= n; i++){
            g[i].clear();
        }
        for(int i = 1; i < n; i++){
            cin >> edgeU[i] >> edgeV[i];
            g[edgeU[i]].push_back(edgeV[i]);
            g[edgeV[i]].push_back(edgeU[i]);
        }
    }
}
```

```

bool chain = true;
for(int i = 1; i <= n; i++){
    if(g[i].size() > 2){
        chain = false;
    }
}
if(chain){
    if(n == 2){
        cout << "0 1\n";
    } else if(n == 3){
        cout << "1 2\n";
    } else{
        cout << 0 << ' ' << n - 3 << '\n';
    }
    continue;
}
int ans = n, ways = 0;
for(int i = 1; i < n; i++){
    bannedU = edgeU[i];
    bannedV = edgeV[i];
    int left = countLeaves(bannedU);
    int right = countLeaves(bannedV);
    int difference = abs(left - right);
    if(difference < ans){
        ans = difference;
        ways = 1;
    } else if(difference == ans){
        ways++;
    }
}
cout << ans << ' ' << ways << '\n';
}
return 0;
}

```

满分做法：从叶向内汇总原叶子

公开分档对应：1 点 $n \leq 100$ 与 2 点 $n \leq 1000$ 用上面的逐边 BFS；3 点是长链，可用该代码中的链特判，前三点共 30 分。4 点虽保证最优边唯一，但仍需检查候选边，故用本节线性算法；5~10 点一般数据也用本节算法。

1. 读入树并统计原度数，得到原叶子总数； $O(n)$ 时间、空间。
2. 从当前度为 1 的节点入队，向唯一未剥离邻点传递累计原叶子权重；每条边处理一次， $O(n)$ 。处理边时已知一侧原叶子数，另一侧为总数减去它。
3. 若删边端点原度为 2，它在本侧新增为叶子；原度为 1 的点原本已计入，成为孤点仍计一次。对每条边 $O(1)$ 算差与计数。

总时间 $O(n)$ 、空间 $O(n)$ 。队列剥离避免长链递归深度，且每条边恰好在一侧被剥离时计一次。

参考代码

```

#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
vector<int> g[N];
int degreeNow[N], degreeOld[N], leaf[N], que[N];

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while(T--){
        int n;
        cin >> n;
        for(int i = 1; i <= n; i++){
            g[i].clear();
            degreeNow[i] = 0;
            leaf[i] = 0;
        }
        for(int i = 1; i < n; i++){
            int u, v;
            cin >> u >> v;
            g[u].push_back(v);
            g[v].push_back(u);
            degreeNow[u]++;
            degreeNow[v]++;
        }
        int total = 0, head = 0, tail = 0;
        for(int i = 1; i <= n; i++){
            degreeOld[i] = degreeNow[i];
            if(degreeNow[i] == 1){
                leaf[i] = 1;
                total++;
                que[tail++] = i;
            }
        }
        int ans = n, ways = 0;
    }
}

```

```

while(head < tail){
    int u = que[head++];
    if(degreeNow[u] != 1) continue;
    int v = 0;
    for(int j = 0; j < (int)g[u].size(); j++){
        int x = g[u][j];
        if(degreeNow[x] > 0){
            v = x;
            break;
        }
    }
    if(v == 0) continue;
    int left = leaf[u] + (degreeOld[u] == 2);
    int right = total - leaf[u] + (degreeOld[v] == 2);
    int delta = abs(left - right);
    if(delta < ans){
        ans = delta;
        ways = 1;
    }else if(delta == ans){
        ways++;
    }
    leaf[v] += leaf[u];
    degreeNow[u] = 0;
    degreeNow[v]--;
    if(degreeNow[v] == 1) que[tail++] = v;
}
cout << ans << ' ' << ways << '\n';
}
return 0;
}

```

知识点总结

断边后的结构量可先用原树上的可加权重计数，再只修正断边两端；从叶向内剥离能线性取得每条边一侧的权重。